VIT

VELLORE INSTITUTE OF TECHNOLOGY

Fall Semester 2026-27

BCSE303P - OPERATING SYSTEMS LAB

PROJECT REVIEW-1 REPORT

Energy-Aware CPU Scheduling in Linux using sched_ext: Core-Compaction Scheduling for Reduced Energy Consumption

Team Member 1: Mohammed Asif A
Register Number: 24BCE2076
Team Member 2: Konasani Vaibhav
Register Number: 24BCE0419
Class Number: VL2026270103058
Slot: L31 + L32
Date: 27/08/2026

1. TITLE

Energy-Aware CPU Scheduling in Linux using sched_ext: Core-Compaction Scheduling for Reduced Energy Consumption

2. BACKGROUND AND MOTIVATION

CPU scheduling is an essential operating-system function that establishes which task that is ready to run will run on which processor and when. The fair scheduling class in modern Linux is the Earliest Eligible Virtual Deadline First (EEVDF) scheduler, which is designed to be fair and have good latency behavior. [3] Linux sched_ext provides a programmable scheduler class that allows the scheduling policy to be implemented with BPF programs and modified at runtime. It makes it easier to experiment with schedulers than modifying and rebuilding the kernel repeatedly. [2] Scheduling with energy considerations is a classical system problem. Linux Energy Aware Scheduling considers CPU-capacity and energy-model data in CPU selection, but prior work has also considered concentrating work on certain subsets of cores. [4], [6] This project reduces the problem to a light-weight interpretable policy: consolidate work on fewer CPUs

Energy aware scheduling is an existing system problem. Linux Energy Aware Scheduling utilizes CPU capacity and an energy model to decide which CPU to schedule on, prior work on this also experimented with pooling workload to a small number of cores [4] [6]. In this project the problem is reduced to designing a minimal, understandable policy – in which workload should be pooled onto fewer CPUs when lightly loaded, work spread out more as contention grows and experimentally assess the performance/energy trade-off.

3.PROBLEM STATEMENT

Schengen General-purpose schedulers need to achieve a balance of fairness, throughput, response time and global efficiency across heterogeneous workloads and hardware. A policy that spreads the work among available CPUs to avoid performance bottlenecks may, under light load, unnecessarily energize more cores than necessary. This project explores whether a naive load-aware core-compaction policy, applied via sched_ext, can lead to energy savings for some applications, without suffering a penalty in their completion time.

4.OBJECTIVES

Primary Objective: To create and benchmark an energy-efficient, load-aware BPF-based load balancing Linux scheduler and compare its energy-performance characteristics with the standard Linux scheduler.

Specific Objectives:

1. Gain a thorough understanding of the sched_ext programming model, scheduling callbacks, and dispatch queues..
2. Implement and run a basic sched_ext scheduler and set up a reliable experiment environment.Implement core-compaction logic that prefers a smaller set of CPUs during light load.
3. Implement a core-compaction feature which ideally focuses on a smaller number of CPUs during a relatively light load.
4. Implement a load-based aspect to work which tends to distribute across a greater number of CPUs when contention rises.
5. Benchmark the new scheduler against the standard Linux EEVDF scheduler using targeted workload tests.
6. Obtain measurements of CPU usage, scheduling latency, and workload completion times/number of contexts-switches..
7. Conduct a brief additional energy test (bare metal using Intel RAPL) if possible hardware.

5.LITERATURE SURVEY

Three ideas are relevant from the literature: the ability to tune program execution within a flexible programmable scheduler expedites scheduler exploration [1, 2], the EEVDF is the currentLinuxfair-scheduling scheme with stated latency management [3], and selecting between CPU resources or a power-performance allocation (such as core subsetting) balances power efficiency against runtime [4, 6]. In addition, work on the already existing sched_extschedulers has showed that power/performance policies can be

effectively implemented in that environment [5]. We will center our work on an uncomplicated, understandable core-compaction policy and provide reproducibility, instead of touting core-compaction as entirely novel.

The relevant literature falls into three categories: flexible programmable scheduling facilitates scheduler experimentation [1, 2]; EEVDF is the current Linux fair scheduling model that includes specific latency targets [3]; CPU selection or core-subsetting that trade performance for energy [4, 6]. Further, existing sched_ext schedulers show that policy on power/performance is feasible within this interface [5]. For this project, therefore, we seek a specific, tiny, transparent core-compaction policy with an reproducible evaluation - not any sort of core compaction.

Ref.	What it shows	How it relates to the project
[1]	ghOSt (2021) showed that flexible scheduling policies can be delegated outside the traditional kernel scheduler while retaining competitive performance.	Provides research lineage for programmable scheduler experimentation; sched_ext is the practical framework used here.
[2]	Linux sched_ext enables BPF-based custom scheduling policies with dynamic activation and scheduler operations exposed through a defined interface.	Provides the implementation framework for the proposed load-aware scheduler.
[3], [4]	EEVDF addresses fair scheduling and latency, while Energy Aware Scheduling incorporates CPU capacity and energy-model information into scheduling decisions.	Defines the default scheduling baseline and the broader energy-aware scheduling context.

Ref.	What it shows	How it relates to the project
[5]	scx_lavd demonstrates that sched_ext can support more advanced power/performance and topology-related policies.	Shows that the proposed lightweight policy is feasible, while keeping the student project deliberately simpler.
[6]	Energy-aware allocation to selected CPU subsets has been studied as a way to trade performance against energy use.	Motivates the core-compaction idea independently of the sched_ext implementation.

5.RESEARCH GAP AND CONTRIBUTION

Previous work can also be found in energy-aware schedulers, core-subset allocation, and advanced schedext schedulers [4]-[6]. The real world hole this work is exploring is a bit closer to earth: a student-level schedext scheduler that implement a simple load-aware core-compaction policy that is measurable on a repeatable laptop-style workload in comparison to the standard Linux scheduler. The scheduler would desire smaller cpu sets for less load, then spreading jobs around as contention increases. So it's mostly about implementing something then measuring it and if it decreases energy consumption without doing worse than the existing scheduler.

6.SCOPE

Core Project: Implementing load-aware core compaction with a BPF-based sched_ext scheduler and comparing it to Linux's EEVDF/default scheduling with experiments for performance/energy evaluation with representative workloads.
Optional Extensions: Comparing with an existing schedext scheduler (e.g., scxlavd) or implementing hardware-topology-aware CPU grouping in the scheduler, only when core implementation is completed ahead of schedule and additional experiments can be conducted with them.
Not intended within project scope: Implementing a new Linux kernel scheduler from the ground up, changes to existing Linux EEVDF implementation itself, complex DVFS control, application-specific scheduling solutions, deployment into production, extensive optimization across all CPU architectures.

7.EXTENSION OF EXISTING OPERATING SYSTEMS LAB EXPERIMENTS

In addition to CPU scheduling algorithms concepts, sched.h process attributes, process states, concurrent issues, or process management under Linux/Unix, this project expands these lab topics by implementing the classic scheduling policies and a new energy-efficient, custom policy. The custom policy leverages the new sched_ext module as a pluggable, loadable scheduling policy and instead of just simulating it, it measures the real world performance of the schedulers under actual workloads. Moreover, it addresses an energy efficiency issue and measures the system level metrics such as latency, utilization, energy, and throughput for the implemented schedulers.

8.COURSE OUTCOME MAPPING

Core Alignment - CO2: The project produces the designs and evaluation of a CPU scheduler policy and tests its scheduling criteria for latency, completion, utilization and context-switching characteristics. VIT identifies the CO2 output as the \"design scheduling algorithms to compute and compare various scheduling criteria.\" [8].

9. METHODOLOGY

A. Build Steps

Make a reproducible Linux system with sched_ext enabled kernel, build & install BPF/LLVM tool-chain and execute one minimalist SCX sample. Measure a baseline at system state having the default Linux scheduler. Define states representing light, moderate & heavy load and create CPU selection / dispatch logic which would minimize the number of active CPUs in light load and grow the active set under load contention.

Verify system correctness with single-tasking, multi-tasking & mixed loads and shed-down/fallback of the scheduler is correct.

On available bare metal machines measure the Intel RAPL energy values before/after runs of workload repeatedly and measure energy consumed per workload unit.

B. Workloads

We chose three illustrative workloads: (1) a CPU bound load using stress-ng, (2) a real compilation workload and (3) a mixed workload intended to simulate typical activity of browser, editor, and terminal. We will run each workload under both default and proposed schedulers given comparable system conditions.

C. Metrics

Primary performance metrics include CPU utilization, workload execution time, scheduling latency, number of context switches, and energy consumption per workload if the RAPL is present. Secondary performance metrics include the number of CPUs with work available, load balance across CPUs, run-to-run variance, and observerations on scheduler overhead or stability. The main experiment is whether applying core compaction reduces energy without suffering from significantly increased completion time or latency for the test workloads; no specific improvement is taken as a prerequisite.

10. TOOLS AND TECHNOLOGIES

- Operating System / Kernel: Linux with sched_ext support; controlled VM for repeatable functional/performance experiments.
- Scheduler Framework: sched_ext / SCX; BPF programs and libbpf-based tooling.
- Programming and Build: C/BPF, Clang/LLVM, Make/CMake as required by the selected SCX examples.
- Benchmarking: stress-ng, hackbench, compiler workload, time/perf/ftrace or related Linux performance tools as appropriate.
- Energy Measurement: Intel RAPL interface on supported bare-metal hardware.
- Data Processing and Visualization: Python, Pandas, NumPy, Matplotlib.

11. EXPECTED DELIVERABLES

- Working sched_ext scheduler source code with documented core-compaction policy.
- Reproducible setup/build/run instructions.
- Benchmark scripts and collected performance datasets.
- RAPL-based energy dataset where hardware support is available.
- Comparison graphs for default EEVDF versus the proposed scheduler.
- Final technical report, README, and demonstration-ready scheduler load/unload workflow.

12. 8-WEEK PROJECT TIMELINE

Time constraint: The project is intentionally scoped for a two-member team with approximately 2 hours per person per week. Advanced topology/DVFS work is explicitly optional so that the core scheduler and evaluation remain achievable within the eight-week window.

Week	Planned Activity
1	Environment setup, sched_ext verification, toolchain installation and initial literature study.
2	Study sched_ext callbacks, DSQs and scx_simple; run an existing example successfully.
3	Baseline benchmarks using default Linux scheduling; create scripts and measurement procedure.
4	Implement first core-compaction version and validate CPU selection/dispatch behavior.
5	Add load-based compact/spread behavior; debug, stabilize and test edge cases.
6	Run controlled comparison experiments against the default scheduler; collect repeated measurements.
7	Perform supplementary RAPL energy measurements on supported bare metal; analyze trade-offs and robustness.
8	Finalize graphs, report, README and live demonstration; prepare conclusions and future work.

Key checkpoints: End of Week 2 - sched_ext example runs reliably; End of Week 4 - first custom policy runs; End of Week 6 - complete default-vs-custom benchmark set; End of Week 8 - reproducible results, documentation and demo.

13. REFERENCES

[1] J. T. Humphries, N. Natu, A. Chaugule, O. Weisse, B. Rhoden, J. Don, L. Rizzo, O. Rombakh, P. T. Turner, and C. Kozyrakis, “ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling,” Proceedings of SOSP, 2021, pp. 588-604. doi:10.1145/3477132.3483542.

[2] T. Heo et al., “sched_ext: Implement BPF extensible scheduler class,” Linux kernel patch series, June 2024. Linux Kernel Mailing List / sched_ext documentation.

[3] Linux Kernel Documentation, “EEVDF Scheduler,” current scheduler documentation. Available from the Linux kernel documentation project.

[4] Linux Kernel Documentation, “Energy Aware Scheduling,” current scheduler documentation. Available from the Linux kernel documentation project.

[5] sched-ext/scx contributors, “scx_lavd,” sched_ext scheduler documentation and source tree, 2025-2026.

[6] M. Libutti et al., “Co-scheduling tasks on multi-core heterogeneous systems: An energy-aware perspective,” IET Computers & Digital Techniques, 2016. doi:10.1049/iet-cdt.2015.0053.

[8] Vellore Institute of Technology, “BCSE303L Operating Systems,” Course Outcomes: CO2 - Design scheduling algorithms to compute and compare various scheduling criteria; syllabus version 1.0.